Linked Data Structures Templates

Rules Modify outside pointer: use ** or return new root/head. malloc check NULL. pop/remove: 1 success, 0 fail. Delete: save next then free. Queue empty: head=tail=NULL. Tree free: postorder.

Common headers + success style

```
#include <stdio.h>
#include <stdlib.h>
/* return 1 = success, 0 = fail */
```

1 1. Singly Linked List

struct + new node

```
typedef struct node {
    int value;
    struct node *next;
} Node;
Node *new_node(int value)
{
    Node *n = malloc(sizeof *n);
    if (n == NULL) return NULL;
    n->value = value;
    n->next = NULL;
    return n;
}
```

init

```
Node *list_new(void)
{
    return NULL;   // empty list head
}
```

insert front / push front

```
void list_push_front(Node **head, int value)
{
    if (head == NULL) return;
    Node *n = new_node(value);
    if (n == NULL) return;
    n->next = *head;
    *head = n;
}
```

insert back / append

```
void list_append(Node **head, int value)
{
    if (head == NULL) return;
    Node *n = new_node(value);
    if (n == NULL) return;
    if (*head == NULL) {
        *head = n;
        return;
    }
    Node *curr = *head;
    while (curr->next != NULL)
        curr = curr->next;
    curr->next = n;
}
```

search

```
int list_contains(Node *head, int target)
{
    Node *curr = head;
    while (curr != NULL) {
        if (curr->value == target) return 1;
        curr = curr->next;
    }
    return 0;
}
```

remove first matching value

```
int list_remove(Node **head, int target)
{
    if (head == NULL || *head == NULL) return 0;
    Node *curr = *head;
    Node *prev = NULL;
    while (curr != NULL) {
        if (curr->value == target) {
            if (prev == NULL) *head = curr->next;
            else prev->next = curr->next;
            free(curr);
            return 1;
        }
        prev = curr;
        curr = curr->next;
    }
    return 0;
}
```

pop front

```
int list_pop_front(Node **head, int *out)
{
    if (head == NULL || *head == NULL || out ==
    NULL)
        return 0;
    Node *tmp = *head;
    *out = tmp->value;
    *head = tmp->next;
    free(tmp);
    return 1;
}
```

free

```
void list_free(Node *head)
{
    Node *curr = head;
    while (curr != NULL) {
        Node *next = curr->next;
        free(curr);
        curr = next;
    }
}
```

2 2. Stack using Linked List

struct + init

```
typedef struct stack {
    Node *top;
} Stack;
Stack *stack_new(void)
{
    Stack *s = malloc(sizeof *s);
    if (s == NULL) return NULL;
    s->top = NULL;
    return s;
}
```

push

```
int stack_push(Stack *s, int value)
{
    if (s == NULL) return 0;
    Node *n = new_node(value);
    if (n == NULL) return 0;
    n->next = s->top;
    s->top = n;
    return 1;
}
```

pop

```
int stack_pop(Stack *s, int *out)
{
    if (s == NULL || s->top == NULL || out ==
    NULL)
        return 0;
    Node *tmp = s->top;
    *out = tmp->value;
    s->top = tmp->next;
    free(tmp);
    return 1;
}
```

peek + free

```
int stack_peek(Stack *s, int *out)
{
    if (s == NULL || s->top == NULL || out ==
    NULL)
        return 0;
    *out = s->top->value;
    return 1;
}
void stack_free(Stack *s)
{
    if (s == NULL) return;
    list_free(s->top);
    free(s);
}
```

Rule Stack = LIFO. Push/pop at top. Empty stack: top == NULL.

3 3. Queue using Linked List

struct + init

```
typedef struct queue {
    Node *head;
    Node *tail;
} Queue;
Queue *queue_new(void)
{
    Queue *q = malloc(sizeof *q);
    if (q == NULL) return NULL;
    q->head = NULL;
    q->tail = NULL;
    return q;
}
```

enqueue back

```
int enqueue(Queue *q, int value)
{
    if (q == NULL) return 0;
    Node *n = new_node(value);
    if (n == NULL) return 0;
    if (q->tail == NULL) {        // empty queue
        q->head = n;
        q->tail = n;
    } else {
        q->tail->next = n;
        q->tail = n;
    }
    return 1;
}
```

dequeue front

```
int dequeue(Queue *q, int *out)
{
    if (q == NULL || q->head == NULL || out ==
    NULL)
        return 0;
    Node *tmp = q->head;
    *out = tmp->value;
    q->head = tmp->next;
    if (q->head == NULL) {        // now empty
        q->tail = NULL;
    }
    free(tmp);
    return 1;
}
```

peek + free

```
int queue_peek(Queue *q, int *out)
{
    if (q == NULL || q->head == NULL || out ==
    NULL)
        return 0;
    *out = q->head->value;
    return 1;
}
void queue_free(Queue *q)
{
    if (q == NULL) return;
    list_free(q->head);
    free(q);
}
```

Rule Queue = FIFO. Enqueue at tail; dequeue from head. If empty, both head and tail are NULL.

4 4. Deque using Doubly Linked List

struct + new node

```
typedef struct dnode {
    int value;
    struct dnode *prev;
    struct dnode *next;
} DNode;
typedef struct deque {
    DNode *front;
    DNode *back;
} Deque;
DNode *new_dnode(int value)
{
    DNode *n = malloc(sizeof *n);
    if (n == NULL) return NULL;
    n->value = value;
    n->prev = NULL;
    n->next = NULL;
    return n;
}
```

init

```
Deque *deque_new(void)
{
    Deque *dq = malloc(sizeof *dq);
    if (dq == NULL) return NULL;
    dq->front = NULL;
    dq->back = NULL;
    return dq;
}
```

push front

```
int deque_push_front(Deque *dq, int value)
{
    if (dq == NULL) return 0;
    DNode *n = new_dnode(value);
    if (n == NULL) return 0;
    if (dq->front == NULL) {
        dq->front = n;
        dq->back = n;
    } else {
        n->next = dq->front;
        dq->front->prev = n;
        dq->front = n;
    }
    return 1;
}
```

push back

```
int deque_push_back(Deque *dq, int value)
{
    if (dq == NULL) return 0;
    DNode *n = new_dnode(value);
    if (n == NULL) return 0;
    if (dq->back == NULL) {
        dq->front = n;
        dq->back = n;
    } else {
        n->prev = dq->back;
        dq->back->next = n;
        dq->back = n;
    }
    return 1;
}
```

pop front

```
int deque_pop_front(Deque *dq, int *out)
{
    if (dq == NULL || dq->front == NULL || out
    == NULL)
        return 0;
    DNode *tmp = dq->front;
    *out = tmp->value;
    dq->front = tmp->next;
    if (dq->front == NULL) dq->back = NULL;
    else dq->front->prev = NULL;
    free(tmp);
    return 1;
}
```

pop back

```
int deque_pop_back(Deque *dq, int *out)
{
    if (dq == NULL || dq->back == NULL || out ==
    NULL)
        return 0;
    DNode *tmp = dq->back;
    *out = tmp->value;
    dq->back = tmp->prev;
    if (dq->back == NULL) dq->front = NULL;
    else dq->back->next = NULL;
    free(tmp);
    return 1;
}
```

free

```
void deque_free(Deque *dq)
{
    if (dq == NULL) return;
    DNode *curr = dq->front;
    while (curr != NULL) {
        DNode *next = curr->next;
        free(curr);
        curr = next;
    }
    free(dq);
}
```

Rule Deque supports push/pop at both ends. Maintain prev and next. If empty: front=back=NULL.

5 5. Binary Tree

struct + new node

```
typedef struct tree_node {
    int value;
    struct tree_node *left;
    struct tree_node *right;
} TreeNode;
TreeNode *new_tree_node(int value)
{
    TreeNode *n = malloc(sizeof *n);
    if (n == NULL) return NULL;
    n->value = value;
    n->left = NULL;
    n->right = NULL;
    return n;
}
```

init + manual connect

```
TreeNode *tree_new(void)
{
    return NULL;          // empty tree root
}
// example: build links manually
// root->left = new_tree_node(1);
// root->right = new_tree_node(2);
```

preorder traversal

```
void preorder(TreeNode *root)
{
    if (root == NULL) return;
    printf("%d\n", root->value);  // root
    preorder(root->left);         // left
    preorder(root->right);        // right
}
```

inorder traversal

```
void inorder(TreeNode *root)
{
    if (root == NULL) return;
    inorder(root->left);          // left
    printf("%d\n", root->value);  // root
    inorder(root->right);         // right
}
```

postorder traversal + free

```
void postorder(TreeNode *root)
{
    if (root == NULL) return;
    postorder(root->left);        // left
    postorder(root->right);       // right
    printf("%d\n", root->value);  // root
}
void tree_free(TreeNode *root)
{
    if (root == NULL) return;
    tree_free(root->left);
    tree_free(root->right);
    free(root);                   // postorder
    free
}
```

Rule Tree has no ordering rule. Traversal: preorder=root-left-right; inorder=left-root-right; postorder=left-right-root. Free tree by postorder.

6 6. Binary Search Tree BST

Property For every node: left subtree values < root < right subtree values. Inorder prints sorted order.

init

```
TreeNode *bst_new(void)
{
    return NULL;          // empty BST root
}
```

insert

```
TreeNode *bst_insert(TreeNode *root, int value)
{
    if (root == NULL)
        return new_tree_node(value);
    if (value < root->value)
        root->left = bst_insert(root->left,
        value);
    else if (value > root->value)
        root->right = bst_insert(root->right,
        value);
    return root;
}
```

insert wrapper with double pointer

```
void bst_insert_inplace(TreeNode **root, int
value)
{
    if (root == NULL) return;
    *root = bst_insert(*root, value);
}
```

search

```
int bst_search(TreeNode *root, int target)
{
    if (root == NULL) return 0;
    if (target == root->value) return 1;
    if (target < root->value)
        return bst_search(root->left, target);
    return bst_search(root->right, target);
}
```

find min / max

```
TreeNode *bst_find_min(TreeNode *root)
{
    if (root == NULL) return NULL;
    while (root->left != NULL)
        root = root->left;
    return root;
}
TreeNode *bst_find_max(TreeNode *root)
{
    if (root == NULL) return NULL;
    while (root->right != NULL)
        root = root->right;
    return root;
}
```

delete / remove

```
TreeNode *bst_delete(TreeNode *root, int target)
{
    if (root == NULL) return NULL;
    if (target < root->value) {
        root->left = bst_delete(root->left,
        target);
    } else if (target > root->value) {
        root->right = bst_delete(root->right,
        target);
    } else {
        if (root->left == NULL) {
            TreeNode *right = root->right;
            free(root);
            return right;
        }
        if (root->right == NULL) {
            TreeNode *left = root->left;
            free(root);
            return left;
        }
        TreeNode *succ =
        bst_find_min(root->right);
        root->value = succ->value;
        root->right = bst_delete(root->right,
        succ->value);
    }
    return root;
}
```

delete wrapper + sorted output

```
void bst_delete_inplace(TreeNode **root, int
target)
{
    if (root == NULL) return;
    *root = bst_delete(*root, target);
}
void bst_print_sorted(TreeNode *root)
{
    inorder(root);        // BST inorder =
    increasing order
}
```

Rule BST delete: 0 child -> free/NULL; 1 child -> return child; 2 children -> replace with inorder successor (min of right subtree), then delete successor.

7 7. Files: open/read/write + select/epoll

low-level file I/O headers

```
#include <fcntl.h>      // open flags
#include <unistd.h>     // read write close
#include <sys/select.h> // select
#include <sys/epoll.h>  // epoll Linux
#include <errno.h>
```

open / close

```
int fd = open("input.txt", O_RDONLY);
if (fd == -1) {
    perror("open");
    return 1;
}
// use fd ...
if (close(fd) == -1) {
    perror("close");
}
```

read loop

```
char buf[1024];
ssize_t n;
while ((n = read(fd, buf, sizeof buf)) > 0) {
    // n = actual bytes read
    write(STDOUT_FILENO, buf, n);
}
if (n == -1) {
    perror("read");
}
```

write all bytes

```
int write_all(int fd, const char *buf, size_t
len)
{
    size_t done = 0;
    while (done < len) {
        ssize_t n = write(fd, buf + done, len -
        done);
        if (n <= 0) return 0;
        done += n;
    }
    return 1;
}
```

select template

```
fd_set set;
FD_ZERO(&set);
FD_SET(fd, &set);
int maxfd = fd;
int ready = select(maxfd + 1, &set, NULL, NULL,
NULL);
if (ready == -1) {
    perror("select");
} else if (FD_ISSET(fd, &set)) {
    // fd is ready to read
    read(fd, buf, sizeof buf);
}
```

epoll template

```
int ep = epoll_create1(0);
struct epoll_event ev;
ev.events = EPOLLIN;
ev.data.fd = fd;
epoll_ctl(ep, EPOLL_CTL_ADD, fd, &ev);
struct epoll_event events[16];
int n = epoll_wait(ep, events, 16, -1);
for (int i = 0; i < n; i++) {
    int rfd = events[i].data.fd;
    read(rfd, buf, sizeof buf);
}
close(ep);
```

Rule fd: 0 stdin, 1 stdout, 2 stderr. read returns actual bytes read; 0 means EOF; -1 error. write may write fewer bytes, so loop until done. select uses maxfd+1. epoll is better for many fds.

COMP2017 Back Side - C Pointers, Memory Model, Synchronisation + IPC Pattern: draw ownership first, then code. Return 1 success, 0 fail. Check NULL/malloc/system calls.

1. Memory Diagram Method

How to draw C memory diagrams

1. Write regions: text/code, data, bss, heap, stack.
2. Put global/static variables in data/bss.
3. Put each function call as a stack frame.
4. Local variables go inside stack frame.
5. malloc objects go on heap.
6. Pointer variable stores an address; arrow points to target.
7. Array name is not a separate pointer variable; array elements are contiguous.
8. Mark strings with final '\0'.
9. Mark invalid pointers after free or function return.
10. Ownership rule: whoever mallocs must eventually free.

Memory regions quick map

text/code	machine instructions
data	initialised global/static
bss	zero/uninitialised global/static
heap	malloc/calloc/realloc objects
stack	locals, params, return addr

stack vs heap example

```
int global = 5;         // data
static int s;           // bss
void f(void) {
    int x = 10;         // stack
    int arr[3];         // stack array
    int *p = malloc(sizeof *p); // heap int
    *p = 7;
    free(p);
}
```

Drawing pointers

`int *p = &x;` means p is a stack variable storing address of x. Draw box p with arrow to x.
`*p` means value at target.
`&p` means address of pointer variable itself.

pointer basics

```
int x = 5;
int *p = &x;
*p = 9;    // changes x
printf("%d", x); // 9
```

Arrays in memory

```
int a[3] = {1,2,3};
```

Draw 3 contiguous boxes: a[0], a[1], a[2]. a in expressions often becomes &a[0], but sizeof(a) is whole array.

2. Pointers with Arrays and Strings

array length

```
int a[10];
size_t n = sizeof(a) / sizeof(a[0]);
// only works in same scope for real array
```

array parameter becomes pointer

```
void f(int a[]) {       // same as int *a
    sizeof(a);          // size of pointer, not
    array
}
```

pointer arithmetic

```
int a[3] = {10,20,30};
int *p = a;
printf("%d", *(p + 1)); // 20
// a[i] == *(a + i)
```

2D array parameter

```
void print(int a[][4], int rows) {
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < 4; j++)
            printf("%d", a[i][j]);
}
```

pointer array vs array pointer

```
int *pa[10];    // array of 10 int pointers
int (*ap)[10];  // pointer to array of 10 ints
```

string array vs string pointer

```
char s1[] = "abc";   // mutable array: a b c \0
char *s2 = "abc";    // points to string
literal; do not edit
s1[0] = 'A';         // ok
// s2[0] = 'A';      // undefined behaviour
```

safe string input

```
char buf[100];
if (fgets(buf, sizeof buf, stdin) != NULL) {
    buf[strcspn(buf, "\n")] = '\0';
}
```

copy string with malloc

```
char *copy_string(const char *s) {
    char *c = malloc(strlen(s) + 1);
    if (c == NULL) return NULL;
    strcpy(c, s);           // copies final \0 too
    return c;
}
```

argv template

```
int main(int argc, char **argv) {
    for (int i = 0; i < argc; i++)
        printf("%s\n", argv[i]);
    return 0;
}
```

3. Enums, Structs, Bitfields, Unions

enum memory

```
enum State { OFF, ON, ERROR = 9 };
enum State s = ON;   // usually stored as int
```

struct layout + padding

```
typedef struct point {
    char c;     // 1 byte
    int x;      // may need padding before x
    double d;   // aligned address
} Point;
// sizeof(Point) >= sum of fields due to padding
```

Struct memory rule

Fields appear in declared order, but compiler may insert padding for alignment. Use sizeof, not manual sum. Reordering fields can reduce padding.

struct pointer

```
Point p;
Point *q = &p;
p.x = 1;
q->x = 2;       // same as (*q).x = 2
```

bitfields

```
typedef struct flags {
    unsigned int read  : 1;
    unsigned int write : 1;
    unsigned int mode  : 3;
} Flags;
```

Bitfield rule

Bitfields pack small integer fields into bits. Layout can be implementation-dependent. Good for flags, not portable binary file formats.

union memory

```
typedef union data {
    int i;
    float f;
    char c[4];
} Data;
Data d;
d.i = 123;
// d.f shares same memory; only last written
member is valid idea
```

Union rule

All fields share the same address. sizeof(union) is at least the size of its largest member plus possible padding.

4. Signals

signal handler

```
#include <signal.h>
#include <unistd.h>
volatile sig_atomic_t stop = 0;
void handler(int sig) {
    stop = 1;               // async-signal-safe
    style
}
int main(void) {
    signal(SIGINT, handler);
    while (!stop) { }
    write(1, "bye\n", 4);
}
```

Common signals

SIGINT Ctrl-C. SIGTERM request terminate. SIGKILL cannot be caught. SIGSEGV invalid memory. SIGCHLD child ended.

send signal

```
#include <signal.h>
kill(pid, SIGTERM);
raise(SIGINT);
```

5. Processes

fork template

```
pid_t pid = fork();
if (pid < 0) {
    perror("fork"); exit(1);
} else if (pid == 0) {
    // child
    exit(0);
} else {
    // parent: pid is child pid
    wait(NULL);
}
```

exec template

```
execlp("ls", "ls", "-l", NULL);
perror("exec");
exit(1);        // only runs if exec failed
```

wait status

```
int status;
pid_t c = wait(&status);
if (WIFEXITED(status)) {
    int code = WEXITSTATUS(status);
}
```

Process rules

fork duplicates process. Parent and child have separate virtual address spaces. File descriptors are inherited. wait prevents zombies. exec replaces current program.

6. IPC: pipes, FIFO, shared memory

pipe parent -> child

```
int fd[2];
pipe(fd);                   // before fork
pid_t pid = fork();
if (pid == 0) {
    close(fd[1]);           // child reads
    char buf[100];
    ssize_t n = read(fd[0], buf, sizeof buf);
    close(fd[0]);
    exit(0);
} else {
    close(fd[0]);           // parent writes
    char *msg = "hello";
    write(fd[1], msg, strlen(msg) + 1);
    close(fd[1]);
    wait(NULL);
}
```

dup2 pipe into exec

```
int fd[2]; pipe(fd);
if (fork() == 0) {
    dup2(fd[1], STDOUT_FILENO);
    close(fd[0]); close(fd[1]);
    execlp("ls", "ls", NULL);
    perror("exec"); exit(1);
}
close(fd[1]);
// parent reads command output from fd[0]
```

FIFO named pipe

```
mkfifo("fifo", 0666);
int fd = open("fifo", O_WRONLY);
write(fd, "hi", 3);
close(fd);
int rd = open("fifo", O_RDONLY);
char buf[10];
read(rd, buf, sizeof buf);
close(rd);
```

shared memory with mmap

```
int *shared = mmap(NULL, sizeof *shared,
    PROT_READ | PROT_WRITE,
    MAP_SHARED | MAP_ANONYMOUS,
    -1, 0);
if (shared == MAP_FAILED) exit(1);
*shared = 0;
if (fork() == 0) {
    (*shared)++;
    exit(0);
}
wait(NULL);
munmap(shared, sizeof *shared);
```

IPC rules

pipe: related processes, fd[0] read, fd[1] write. Close unused ends. FIFO: named pipe for unrelated processes. Shared memory: fastest, but needs synchronisation.

7. Threads

pthread create/join

```
#include <pthread.h>
void *worker(void *arg) {
    int *x = arg;
    printf("%d\n", *x);
    return NULL;
}
int main(void) {
    pthread_t t;
    int v = 5;
    pthread_create(&t, NULL, worker, &v);
    pthread_join(t, NULL);
}
```

passing safe thread args

```
typedef struct arg {
    int id;
    int *arr;
} Arg;
Arg args[N];
for (int i = 0; i < N; i++) {
    args[i].id = i;
    pthread_create(&t[i], NULL, worker,
    &args[i]);
}
```

Thread memory rule

Threads share heap, globals, file descriptors. Each thread has its own stack and registers. Shared data needs synchronisation.

8. Applying Synchronisation Primitives

How to choose primitive

Mutex: protect one critical section.
Semaphore: count available resources or limit capacity.
Condition variable: sleep until a condition becomes true.
Atomic/reduction: avoid shared write by using local results, combine later.

mutex critical section

```
pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;
int counter = 0;
void *worker(void *arg) {
    pthread_mutex_lock(&lock);
    counter++;              // not atomic without
    lock
    pthread_mutex_unlock(&lock);
    return NULL;
}
```

semaphore template

```
#include <semaphore.h>
sem_t slots;
sem_init(&slots, 0, 3);     // 3 resources
sem_wait(&slots);           // take one, may
block
// use resource
sem_post(&slots);           // release one
sem_destroy(&slots);
```

condition variable wait

```
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t cv = PTHREAD_COND_INITIALIZER;
int ready = 0;
pthread_mutex_lock(&m);
while (!ready) {            // always while, not
if
    pthread_cond_wait(&cv, &m);
}
// condition true, do work
pthread_mutex_unlock(&m);
```

condition variable signal

```
pthread_mutex_lock(&m);
ready = 1;
pthread_cond_signal(&cv);   // or broadcast
pthread_mutex_unlock(&m);
```

producer-consumer skeleton

```
// shared: queue q, mutex m, cond
not_empty/not_full
pthread_mutex_lock(&m);
while (queue_full(q))
    pthread_cond_wait(&not_full, &m);
enqueue(q, item);
pthread_cond_signal(&not_empty);
pthread_mutex_unlock(&m);

pthread_mutex_lock(&m);
while (queue_empty(q))
    pthread_cond_wait(&not_empty, &m);
item = dequeue(q);
pthread_cond_signal(&not_full);
pthread_mutex_unlock(&m);
```

parallel reduction pattern

```
typedef struct arg { int lo, hi; int *a; long
sum; } Arg;
void *sum_worker(void *p) {
    Arg *x = p;
    long local = 0;
    for (int i = x->lo; i < x->hi; i++)
        local += x->a[i];
    x->sum = local;         // each thread writes
    own slot
    return NULL;
}
// join all, then total += args[i].sum
```

thread pool core idea

```
typedef struct task {
    void (*fn)(void *);
    void *arg;
    struct task *next;
} Task;
// workers loop:
lock(m);
while (queue_empty(q) && !shutdown)
    cond_wait(cv, m);
task = pop(q);
unlock(m);
if (task) task->fn(task->arg);
```

9. Deadlock Avoidance Templates

Deadlock rule checklist

Deadlock needs: mutual exclusion + hold-and-wait + no preemption + circular wait. Break circular wait by always locking in one global order. Keep critical sections short. Never wait on condition while holding unrelated locks.

lock ordering pattern

```
// Always acquire locks in the same order
everywhere.
pthread_mutex_t A = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t B = PTHREAD_MUTEX_INITIALIZER;
void safe_work(void) {
    pthread_mutex_lock(&A);     // lock
    lower/order-first
    pthread_mutex_lock(&B);
    // critical section using A and B
    pthread_mutex_unlock(&B);   // unlock
    reverse order
    pthread_mutex_unlock(&A);
}
```

trylock to avoid circular wait

```
while (1) {
    pthread_mutex_lock(&A);
    if (pthread_mutex_trylock(&B) == 0) {
        // got both locks
        break;
    }
    pthread_mutex_unlock(&A);   // release and
    retry later
    sched_yield();              // or
    sleep/backoff
}
// use A and B
pthread_mutex_unlock(&B);
pthread_mutex_unlock(&A);
```

single cleanup path for unlock

```
int ok = 0;
pthread_mutex_lock(&m);
if (bad_condition)
    goto cleanup;
// work while locked
ok = 1;
cleanup:
pthread_mutex_unlock(&m);
return ok;
```

Deadlock exam traps

Do not lock A then B in one thread and B then A in another. Do not return early while holding a mutex. Cond wait releases only its own mutex, not other locks. Use while for condition wait.

10. Process Two-way Communication

two pipes: parent <-> child

```
int p2c[2], c2p[2];
pipe(p2c); pipe(c2p);           // before fork
pid_t pid = fork();
if (pid == 0) {
    close(p2c[1]);              // child reads
    from parent
    close(c2p[0]);              // child writes
    to parent
    char buf[100];
    read(p2c[0], buf, sizeof buf);
    // process buf ...
    write(c2p[1], "ok", 3);   // include final
    \0 if string
    close(p2c[0]);
    close(c2p[1]);
    exit(0);
} else {
    close(p2c[0]);              // parent writes
    to child
    close(c2p[1]);              // parent reads
    from child
    write(p2c[1], "hello", 6);
    close(p2c[1]);              // signal EOF to
    child
    char reply[100];
    read(c2p[0], reply, sizeof reply);
    close(c2p[0]);
    wait(NULL);
}
```

parent sends many requests

```
// parent: write requests, then close write end
for (int i = 0; i < n; i++) {
    write(p2c[1], &req[i], sizeof req[i]);
}
close(p2c[1]);                  // child read
loop can finish
// child: read until EOF
Request r;
while (read(p2c[0], &r, sizeof r) == sizeof r) {
    Response out = handle(r);
    write(c2p[1], &out, sizeof out);
}
```

Two-way IPC rules

Need two pipes for full duplex. Close unused ends immediately. Close write end when done so other side gets EOF. Avoid both processes blocking while both try to read first. For structs, read/write exactly sizeof struct; for strings, include '\0' or send length.

11. Exam Traps

Common bugs

- forgetting malloc/system-call NULL or -1 check.
- using sizeof(pointer) instead of sizeof(array).
- returning address of local stack variable.
- editing string literal through char*.
- missing final '\0' for strings.
- pipe read blocks because write end still open.
- fork without wait creates zombie.
- exec success never returns.
- counter++ is not atomic.
- condition wait uses if instead of while.
- unlocking mutex on wrong path / early return.
- shared memory without synchronisation can race.

Minimal answer patterns

Memory diagram: locate stack/heap/-global, draw arrows, mark ownership/free.
Pointer arrays: array contiguous, pointer stores address, string ends with \0.
Process IPC: pipe before fork, close unused ends, wait.
Threads: find shared variable, protect critical section, join.
Cond var: lock, while condition false wait, update condition, signal.